

Machine de Turing

BOLARD Yassine et MANSO Alexis

Table des matières

1	Introduction	3
1.1	Un peu d'histoire	3
1.2	Le Projet	3
1.3	Fonctionnement d'une Machine de Turing	4
2	Reprise de l'existant.....	6
3	Conception	7
3.1	Cahier des charges	7
3.1.1	Partie 1	7
3.1.2	Partie 2	7
3.1.3	Partie 3	7
3.1.4	À ne pas faire !	7
3.2	Définition des besoins	8
3.2.1	Diagramme bête à corne.....	8
3.2.2	Matrice Moscow.....	8
3.3	Conception Matérielle	9
3.3.1	SFnD	9
3.3.3	Technologies à utiliser.....	11
3.3.4	Spécifications des fonctions et de leurs signaux de communication	13
3.3.5	Les composants	15
3.3.6	Schéma du circuit électronique	16
3.4	Conception Logicielle	16
3.4.1	Module d'exécution de la Machine de Turing.....	16
3.4.2	Module de Parsing.....	19
3.4.3	Gestion des erreurs	23
3.4.4	Amélioration possible	24
4	Synthèse.....	25
5	Remerciements.....	26
	Bibliographie.....	27

1 Introduction

1.1 Un peu d'histoire

La machine de Turing est un modèle théorique fondamental en informatique, introduit en 1936 par le mathématicien britannique Alan Mathison Turing. Dans son article « On Computable Numbers, with an Application to the Entscheidungsproblem », Turing formalisa le concept de calculabilité et posa les bases de ce qui allait devenir l'informatique moderne. Une machine de Turing est une abstraction qui simule la logique d'un algorithme à l'aide d'un ruban infini, d'une tête de lecture/écriture et d'une table de transition définissant les actions à effectuer en fonction de l'état courant et du symbole lu.

Alan Turing (1912 – 1954), figure centrale de l'histoire des sciences, fut non seulement le père de la théorie de la computation mais aussi un cryptanalyste brillant. Il joua un rôle décisif durant la Seconde Guerre mondiale en contribuant à la décryption de la machine allemande Enigma, ce qui permit d'accélérer la fin du conflit. Turing posa également les bases de l'intelligence artificielle avec le célèbre « Test de Turing ». Tragiquement, il fut condamné en 1952 en raison de son homosexualité et mourut deux ans plus tard dans des circonstances controversées. Ce n'est qu'en 2009 qu'une reconnaissance officielle et publique de son rôle essentiel fut prononcée par le gouvernement britannique.

1.2 Le Projet

Le présent document constitue un rapport complet sur la réalisation d'un système embarqué représentant une machine de Turing reposant sur un microcontrôleur PIC24FJ64GA002. Ce système est conçu pour exécuter des algorithmes de transition stockée sur une carte microSD visualiser leur exécution sur une matrice RGB et permettre à l'utilisateur d'interagir avec la machine via des boutons.

Ce projet s'inspire de travaux antérieurs détaillés dans le rapport de Clément ALLEGRE-COMMINGES et Romain BROUARD (2024)

1.3 Fonctionnement d'une Machine de Turing

Le rapport de Clément ALLEGRE-COMMINGES et Romain BROUARD explique correctement le fonctionnement d'une MTU (Machine de Turing) cité ci-dessous :

Une MTU se compose de quatre éléments essentiels :

- Un ruban de taille infinie, divisé en cases.
- Une tête de lecture/écriture (qu'on appellera simplement tête de lecture, même si elle permet également d'écrire sur le ruban)
- Un état interne
- Une table de transition.

Une machine de Turing traite des symboles. L'ensemble des appelé Alphabet. Le ruban permet d'accueillir des symboles qui seront lus et écrits par la tête de lecture. La tête de lecture permet de lire et écrire un symbole. Elle peut se déplacer vers la gauche ou la droite.

Une machine de Turing fonctionne donc de la manière suivante :

1. La tête de lecture lit le symbole qu'elle pointe sur le ruban.
2. En fonction de son état et du symbole lu, la tête de lecture écrit un symbole à la place de celui qu'elle a lu précédemment.
3. Un déplacement est choisi en fonction de l'état de la tête de lecture et du symbole lu.
4. La tête de lecture change d'état.
5. Le ruban se déplace vers la droite ou vers la gauche selon le déplacement choisi précédemment.
6. Puis on recommence depuis le 1.

On peut donc résumer le fonctionnement comme cela : à chaque « cycle », on choisi un symbole à écrire, un nouvel état pour la tête de lecture, et un déplacement, en fonction d'un symbole lu et de l'état actuel. On peut donc définir une fonction de transition, qui va se charger de déterminer l'état futur d'une MTU en fonction de ton état courant. L'ensemble des fonctions de transition permettant de traiter un « mot » peut être représenté sous la forme d'une table de transitions ou d'un graphe de transitions.

Un mot traité est dit accepté si une Machine de Turing s'arrête en état final après l'avoir intégralement traité.

Une MTU possède forcément un nombre fini d'états, et elle a au moins deux états obligatoires : q_0 l'état initial, et F un ensemble d'états d'acceptation.

Une machine de Turing est donc définie par :

- Q un ensemble fini d'états.
- q_0 un état initial tel que $q_0 \in Q$.
- F un ensemble d'états d'acceptation tel que $F \subseteq Q$.
- Γ un ensemble fini de symboles.
- Σ un ensemble fini de symboles d'entrée tel que $\Sigma \subseteq \Gamma$.
- B un symbole de ruban vide tel que $B \in \Gamma \setminus \Sigma$.
- δ une fonction de transition.

Une fonction de transition se formalise donc comme ceci :

$$\delta(q, Z) \rightarrow (p, Y, D)$$

Avec q l'état de la tête de lecture, Z le symbole pointé, p le nouvel état, Y le nouveau symbole et D le déplacement.

Exemple d'une table de transition pour une Machine de Turing acceptant le langage $L = \{a^k b^k \mid k > 0\}$ avec q_0 comme état initial (représenté par une $\rightarrow$), et q_4 comme état d'acceptation (représenté par $*$) :

	symboles				
	a	b	X	Y	Blank
$\rightarrow q_0$	(q1, X, R)			(q3, Y, R)	
q1	(q1, a, R)	(q2, Y, L)		(q1, Y, R)	
q2	(q2, a, L)		(q0, X, R)	(q2, Y, L)	
q3				(q3, Y, R)	(q4, B, R)
* q4					

L'objectif de ce projet va donc être de concevoir une Machine de Turing électronique fonctionnant tel que défini ci-dessus. Pour cela, nous avons repris les composants de l'ancien projets.

2 Reprise de l'existant

Le projet de la Machine de Turing est déjà un projet d'électronique qui est présent depuis deux ans dans la promotion de troisième année en Informatique et Systèmes Intelligents Embarqués de Polytech Tours. Nous sommes donc la troisième équipe à reprendre le projet et ne partons pas de rien. Nous disposons des différents modules commandés par les anciens groupes dont un PIC24FJ64GA002 qui est le microcontrôleur du projet, 8 boutons poussoir qui seront à programmer, un module de lecture de carte microSD, un écran LCD 2 lignes, 2 matrices RGB 8x8 et des modules MCP23017 qui permettent d'ajouter des bus I2C.

Nous avons récupéré le SF1D ainsi que les analyses des fonctions de l'équipe précédente pour le projet.

3 Conception

3.1 Cahier des charges

Le cahier des charges est priorisé en trois parties. La première étant la plus importante, la seconde est facultative mais recommandé et la troisième si on a le temps.

3.1.1 Partie 1

Dans les besoins du projet vu avec l'enseignant, il nous a été demandé de :

- Créer un système capable de dérouler un algorithme via un fichier donné
- Afficher l'état des différents rubans qui auront 16 cases de long
- Afficher la position de la tête de lecture
- Stocker les programmes dans une carte SD

3.1.2 Partie 2

Une fois ces objectifs remplis, il nous a été recommandé d'essayer de :

- Permettre la sélection d'un programme sur l'écran LCD
- Mettre un mode continu / pas à pas pour l'exécution du programme
- Stocker les programmes dans le PIC24

3.1.3 Partie 3

Si tous les objectifs sont remplis et qu'ils nous restent du temps nous pouvons terminer par :

- Ajouter une fonction donnant la possibilité de coder un algorithme directement via la machine
- Initialiser manuellement le ruban et la tête de lecture
- Reset la programmation de la ligne en cours

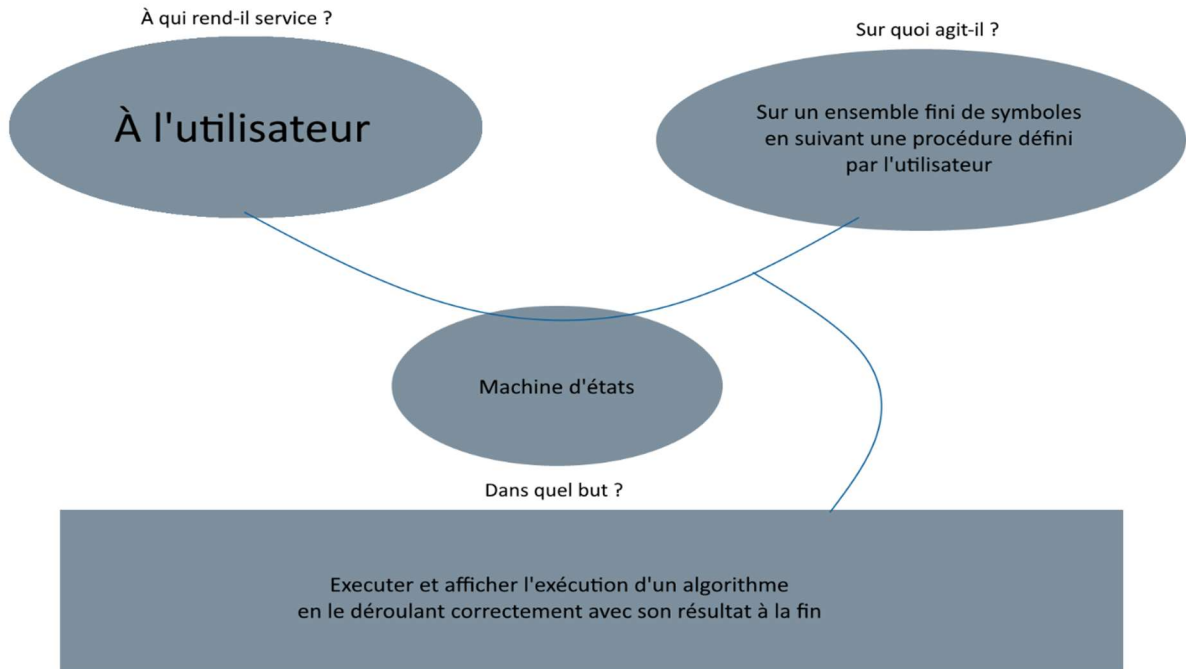
3.1.4 À ne pas faire !

Dans tous ces objectifs, il nous a été précisé une chose à ne pas faire pour améliorer le projet et ne pas faire comme les équipes précédentes :

- Coder l'algorithme en dur dans un programme

3.2 Définition des besoins

3.2.1 Diagramme bête à corne



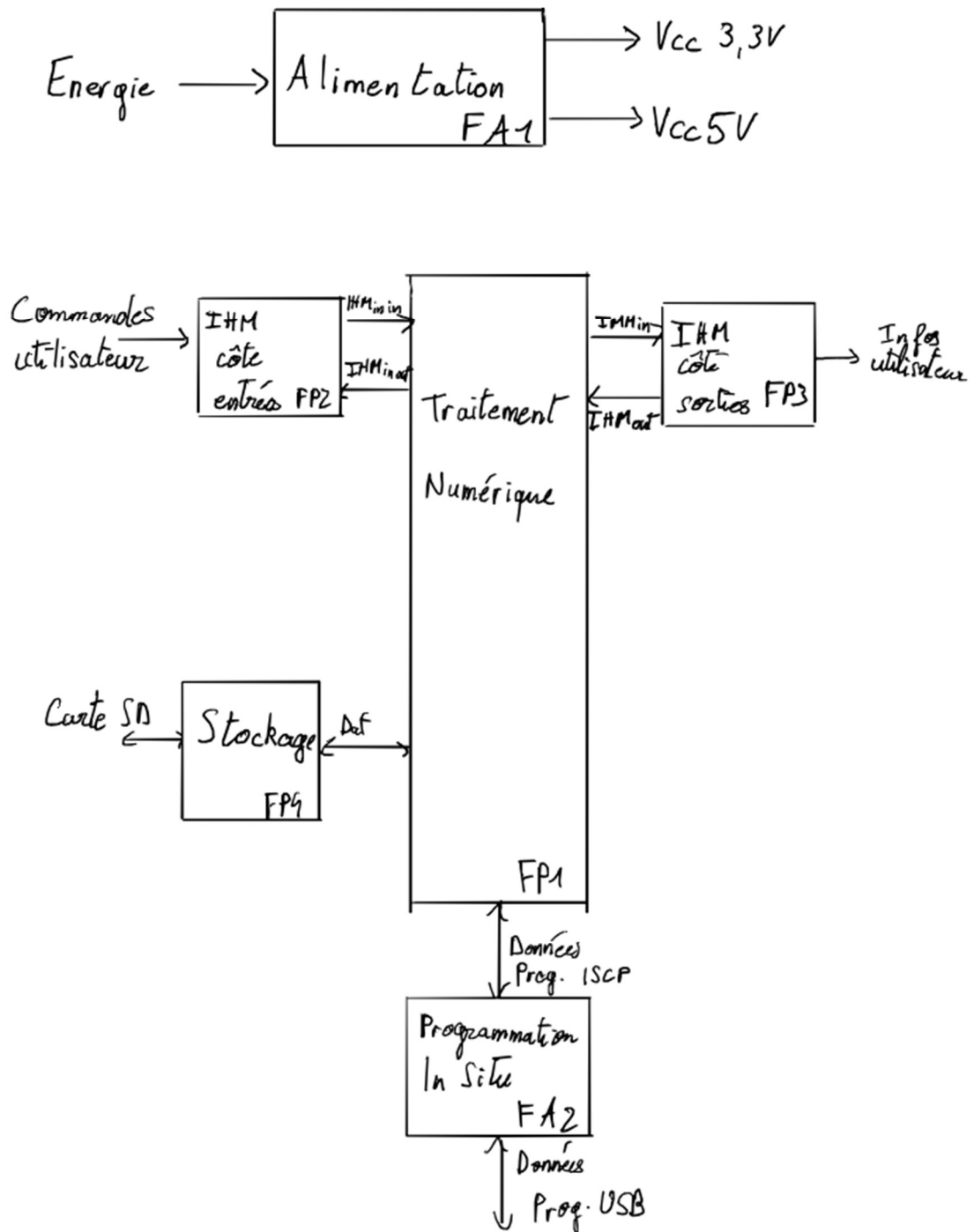
3.2.2 Matrice Moscow

<u>Must have</u> - Machine de Turing capable de dérouler un algorithme via un fichier donné - Affichage de l'état des différents rubans de taille 16 - Affichage de la position de la tête de lecture - Stocker les programmes dans la carte SD	<u>Should Have</u> - Sélection du programme sur l'écran LCD - Mode continu / pas à pas pour l'exécution du programme - Stocker les programmes dans le PIC24
<u>Could Have</u> - Possibilité de coder directement via la machine - Initialisation manuelle du ruban et de la tête de lecture - Reset de la programmation de la ligne en cours	<u>Won't Have</u> - Coder l'algorithme en dur dans un programme

3.3 Conception Matérielle

3.3.1 SFnD

Nous sommes partis du SF1D de l'équipe précédente rajoutant 2 détails, nous avons précisé que la fonction annexe 1 (alimentation) doit bien fournir 2 VCC dont un de 3.3V et un second de 5V et la fonction principale 4 (stockage) communique bien avec une carte SD pour récupérer et transmettre des datas.



Par la suite, nous pouvons rappeler les fonctions principales :

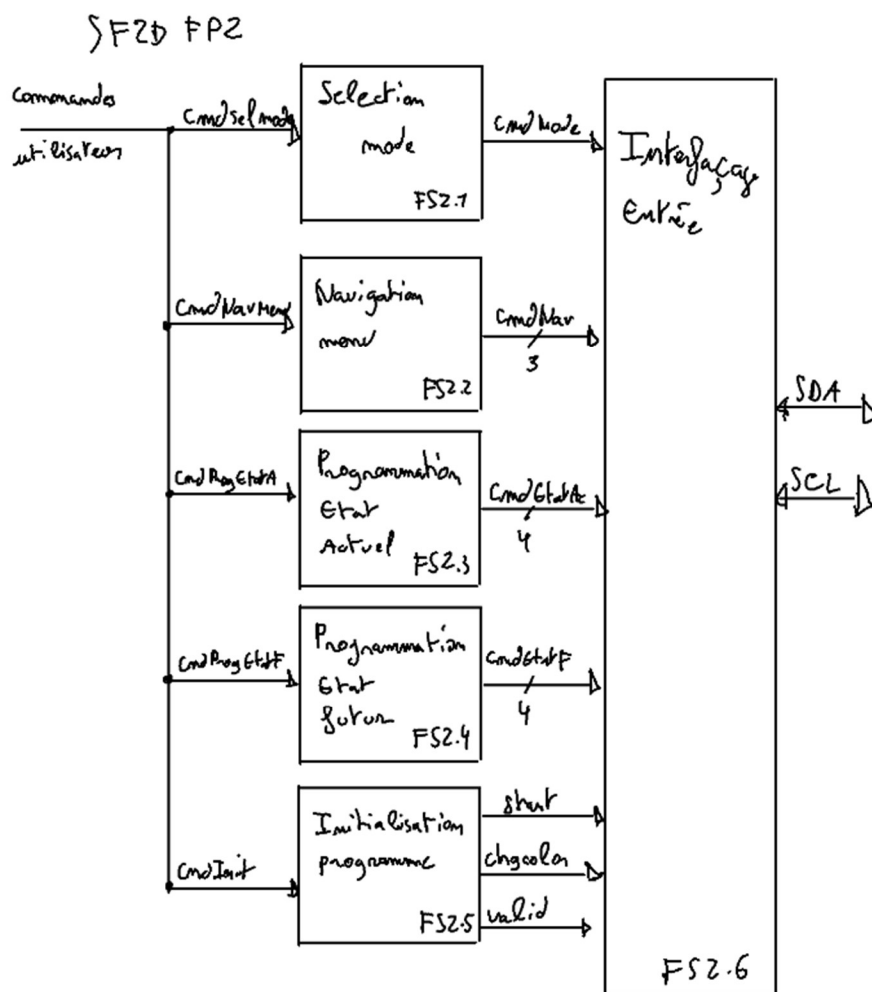
- Le traitement numérique (FP1)
- La gestion des entrées (FP2)
- La gestion des sorties (FP3)
- La gestion du stockage (FP4)

Avec ceci, on ajoute deux fonctions annexes :

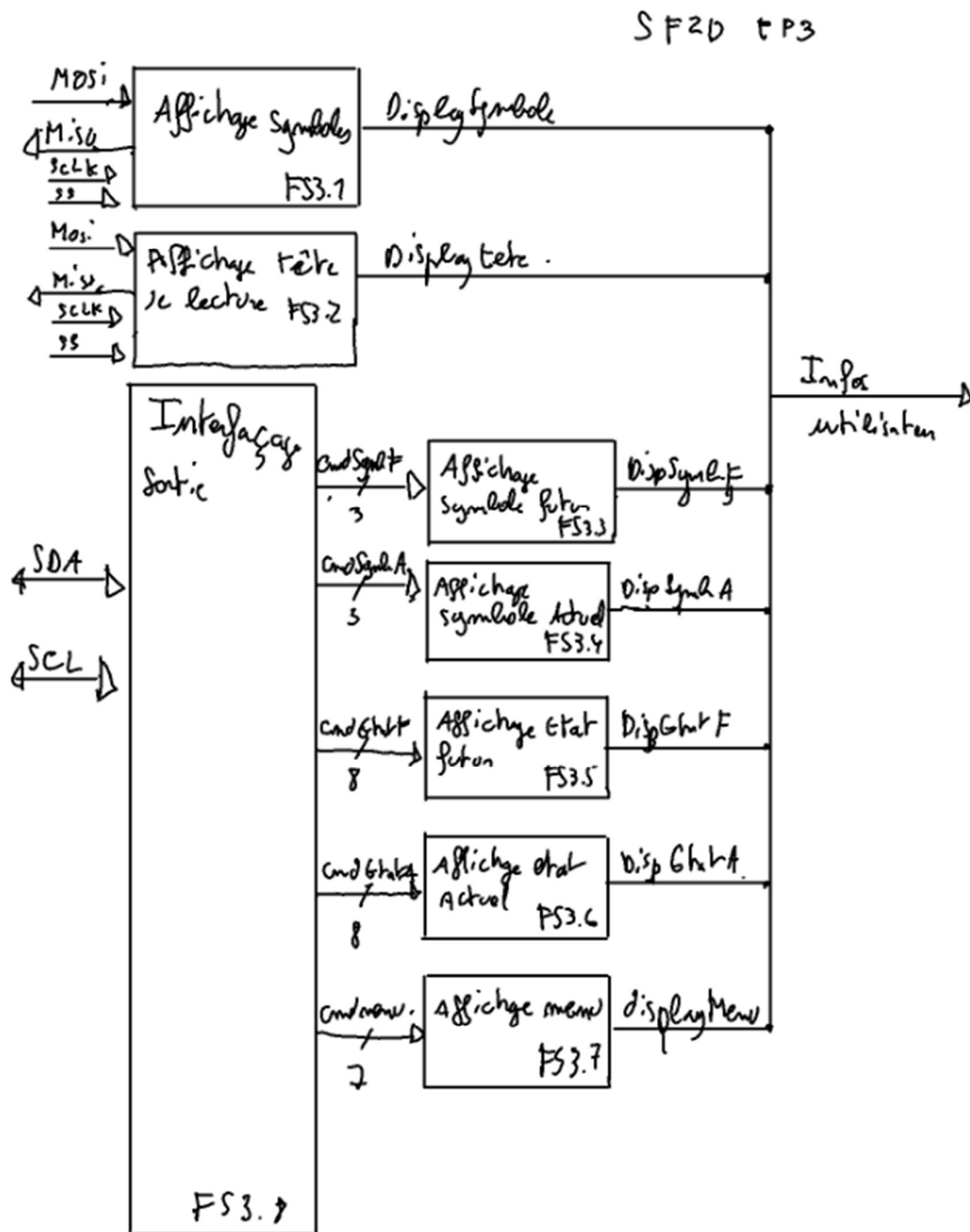
- L'alimentation (FA1) qui va se charger de fournir donc 2 tensions différentes pour alimenter les différents modules
- La programmation in situ (FA2)

Nous sommes restés sur les mêmes programmations de fonctions que l'équipe précédente avec les SF2D de FP2 et FP3

FP2 :



FP3 :



3.3.3 Technologies à utiliser

Pour les technologies de communication, nous partons sur des composants qui communiquent en I2C et en SPI

Bus I²C : Le bus I²C (Inter-Integrated Circuit) est un protocole de communication série synchrone inventé par Philips, conçu pour permettre à plusieurs circuits intégrés de communiquer entre eux à l'aide de seulement deux lignes : une pour l'horloge (SCL) et une pour les données (SDA). Il permet une communication maître/esclave, où un maître peut contrôler un ou plusieurs périphériques esclaves en les adressant individuellement grâce à une adresse unique. Compact, économique en broches et relativement simple à mettre en œuvre, le bus I²C est largement utilisé pour interfacer des capteurs, des mémoires, des afficheurs et d'autres composants dans les systèmes embarqués.

Bus SPI : Le bus SPI (Serial Peripheral Interface) est un protocole de communication série synchrone développé par Motorola, utilisé pour des échanges rapides de données entre un maître et un ou plusieurs esclaves. Il fonctionne à l'aide de quatre lignes : MOSI (Master Out Slave In), MISO (Master In Slave Out), SCLK (horloge) et SS/CS (Slave Select/Chip Select) pour sélectionner l'esclave actif. Contrairement à l'I²C, le SPI permet des transmissions full-duplex (envoi et réception simultanés) et offre des débits plus élevés, au prix d'une consommation plus importante de broches, surtout en cas de multiples périphériques. Il est couramment utilisé pour les mémoires flash, les écrans ou les convertisseurs analogique/numérique.

3.3.4 Spécifications des fonctions et de leurs signaux de communication

Fonction	FP1	Traitements numériques
Description / rôle	Réalise tous les traitements numériques nécessaires au bon fonctionnement du système	
Signaux d'entrée	MISO	
Signaux de sortie	MOSI, SCLK, SS, SCL	
Signaux E/S	SDA, ISCP	

Fonction	FS2.1	Sélection du mode
Description / rôle	Fonction qui sert à sélectionner le mode d'exécution, entre pas à pas et continu	
Signaux d'entrée	CmdSelMode	
Signaux de sortie	CmdMode	
Signaux E/S	NA	

Fonction	FS2.2	Navigation Menu
Description / rôle	Fonction qui sert à naviguer dans le menu	
Signaux d'entrée	CmdNavMenu	
Signaux de sortie	CmdNav<0 :2>	
Signaux E/S	NA	

Fonction	FS2.3	Programmation Etat Actuel
Description / rôle	Fonction qui permet de programmer l'état actuel de la table de transition	
Signaux d'entrée	CmdProgEtatA	
Signaux de sortie	CmdEtatA<0 :3>	
Signaux E/S	NA	

Fonction	FS2.4	Programmation Etat Futur
Description / rôle	Fonction qui permet de programmer l'état futur de la table de transition	
Signaux d'entrée	CmdProgEtatF	
Signaux de sortie	CmdEtatF<0 :3>	
Signaux E/S	NA	

Fonction	FS2.5	Initialisation Programme
Description / rôle	Fonction qui permet d'initialiser les symboles et la position de la tête de lecture	
Signaux d'entrée	CmdInit	
Signaux de sortie	Start, ChgColor, Valid	
Signaux E/S	NA	

Fonction	FS2.6	Interfaçage Entrée
Description / rôle		
Signaux d'entrée	CmdMode, CmdNav<0 :2>, CmdEtatA<0 :3>, CmdEtatF<0 :3>, Start, ChgColor, Valid	
Signaux de sortie	SCL	
Signaux E/S	SDA	

Fonction	FS3.1	Affichage Symboles
Description / rôle	Réalise l'affichage des symboles (représente la matrice)	
Signaux d'entrée	MOSI, SS, SCLK	
Signaux de sortie	MISO, DisplaySymboles	
Signaux E/S	NA	

Fonction	FS3.2	Affichage tête de lecture
Description / rôle	Réalise l'affichage de la tête de lecture.	
Signaux d'entrée	MOSI, SS, SCLK	
Signaux de sortie	MISO, DisplayTete	
Signaux E/S	NA	

Fonction	FS3.3	Affichage Symbole Futur
Description / rôle	Affiche le symbole futur (après transition)	
Signaux d'entrée	CmdSymboleF<0 :2>	
Signaux de sortie	DisplaySymboleF	
Signaux E/S	NA	

Fonction	FS3.4	Affichage Symbole Actuel
Description / rôle	Affiche le symbole courant (avant transition)	
Signaux d'entrée	CmdSymboleA<0 :2>	
Signaux de sortie	DisplaySymboleA	
Signaux E/S	NA	

Fonction	FS3.5	Affichage Etat Futur
Description / rôle	Affiche l'état futur de la tête de lecture (après transition)	
Signaux d'entrée	CmdEtatF<0 :7>	
Signaux de sortie	DisplayEtatF	
Signaux E/S	NA	

Fonction	FS3.6	Affichage Etat Actuel
Description / rôle	Affiche l'état courant de la tête de lecture (avant transition)	
Signaux d'entrée	CmdEtatA<0 :7>	
Signaux de sortie	DisplayEtatA	
Signaux E/S	NA	

Fonction	FS3.7	Affichage Menu
Description / rôle	Affiche le menu	
Signaux d'entrée	CmdMenu<0 :6>	
Signaux de sortie	DisplayMenu	
Signaux E/S	NA	

Fonction	FS3.8	Interfaçage sortie
Description / rôle		
Signaux d'entrée	CmdSymboleF<0 :2>, CmdSymboleA<0 :2>, CmdEtatF<0 :7>, CmdMenu<0 :6>	
Signaux de sortie	SCL	
Signaux E/S	SDA	

Fonction	FP4	Stockage
Description / rôle	Permet le stockage des algorithmes dans une carte SD	
Signaux d'entrée	MOSI, SCLK, SS	
Signaux de sortie	MISO	
Signaux E/S	NA	

Fonction	FA1	Alimentation
Description / rôle	Alimente le système en électricité	
Signaux d'entrée	Energie	
Signaux de sortie	VCC3.3V, VCC5V	
Signaux E/S	NA	

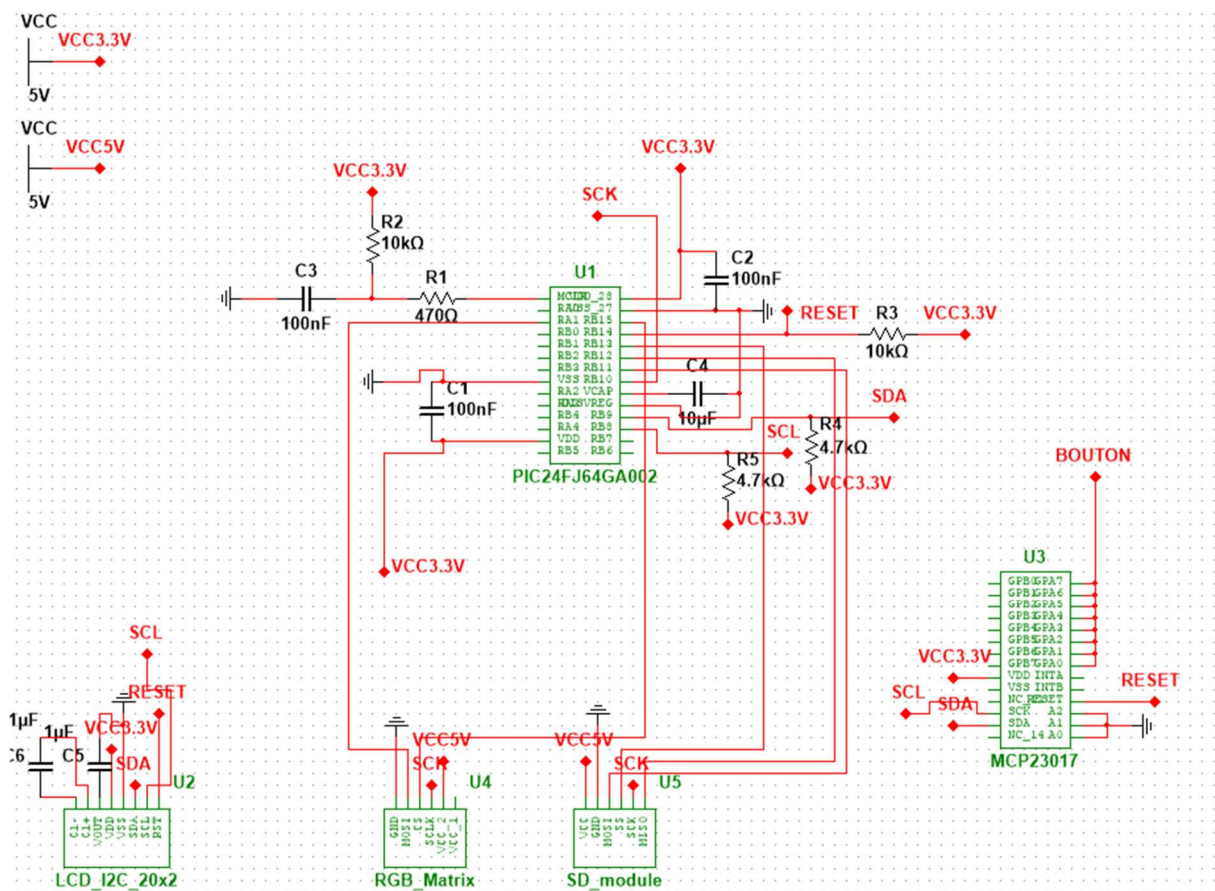
Fonction	FA2	Programmation In Situ
Description / rôle	Permet de charger le programme dans le MCU	
Signaux d'entrée	NA	
Signaux de sortie	NA	
Signaux E/S	Données USB, ISCP	

3.3.5 Les composants

Dans le projet dont nous avons hérité, les composants qu'on nous a fourni sont :

- Le PIC24FJ64GA002 qui sera le cerveau du projet, il dispose de bus I2C ainsi que SPI qui nous permettra de contrôler les différents modules dont nous avons à disposition. Il est alimenté en 3.3V
- Un écran LCD I2C 20x2 de chez NEWHAVEN DISPLAY, modèle NHD-C0220BiZ-FSW-FBW-3V3M. C'est donc un écran qui fonctionne sous bus I2C alimenté en 3.3V qui nous permettra de faire l'écran principale du menu de la machine pour la sélection des programmes
- Un module MCP23017 qui nous permet de rajouter des GPIOs ce qui va nous permettre de connecter nos 8 boutons. Il fonctionne sous bus I2C aussi alimenté en 3.3V
- Un module MicroSD DFR0229 qui fonctionne sous bus SPI alimenté en 5V qui nous permettra de charger les programmes des algorithmes.
- Deux modules RGB Matrix de la marque Sparkfun Electronics fonctionnant sous bus SPI et alimenté en 5V. Avec ces matrices, nous avons la possibilité de les combiner directement entre elles ce qui nous permettra d'avoir finalement une matrice 8x16 RGB. Ces modules permettront de montrer à l'utilisateur l'algorithme qui se déroule.
- Un module d'alimentation de chez Joy-It qui permet de brancher la source d'alimentation à un secteur ou une prise USB-C afin de recevoir deux sources de tensions, une source 3.3V et une source 5V.

3.3.6 Schéma du circuit électronique



3.4 Conception Logicielle

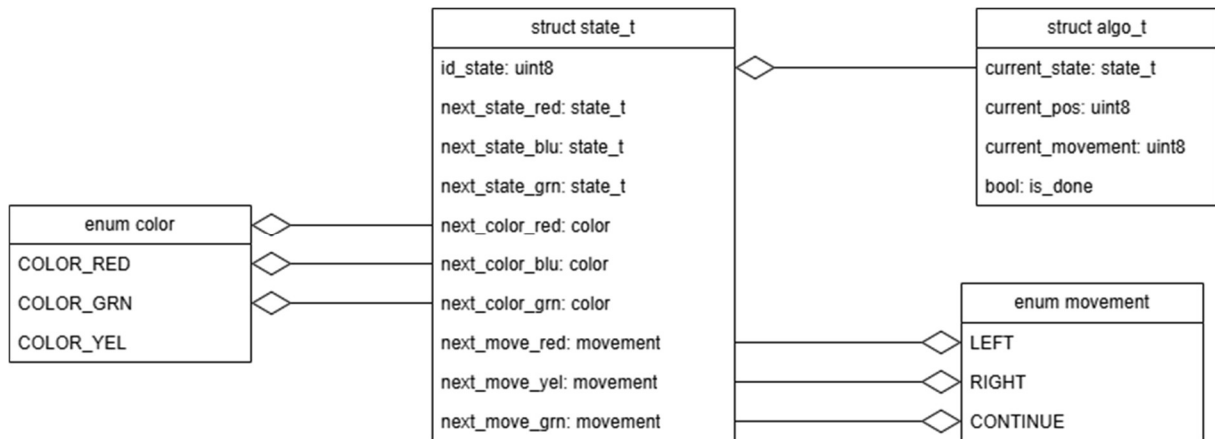
3.4.1 Module d'exécution de la Machine de Turing

3.4.1.1 Objectif

Le module d'algorithme contrôle l'évolution de la machine au fil des transitions :

- Il agit sur les LEDs selon les règles des états courants.
- Il déplace la tête de lecture sur le ruban.
- Il met à jour l'état courant de la machine.

Nous avons donc utilisé la structure state suivante :



3.4.1.2 *Fonction principale*

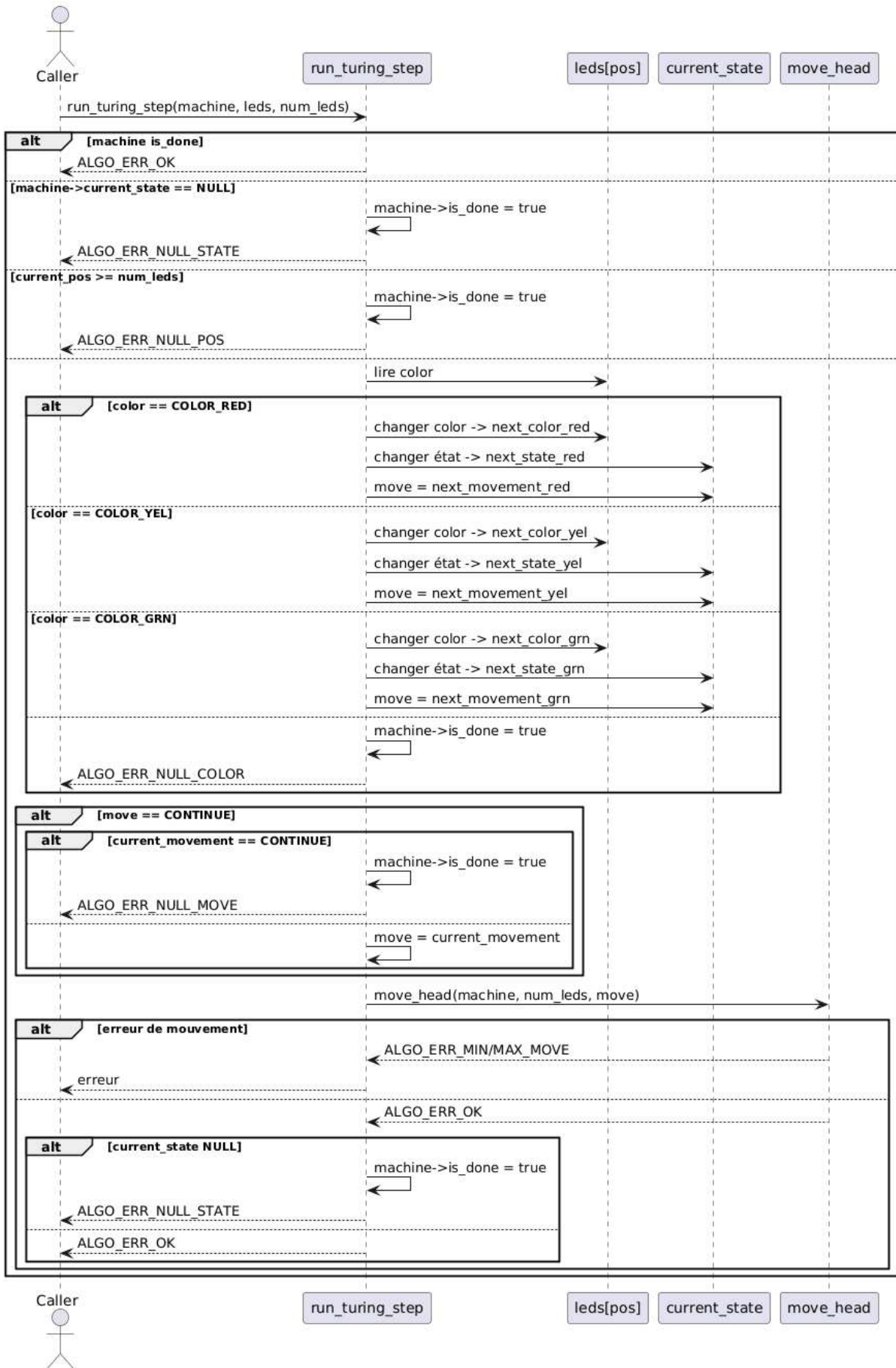
run_turing_step

Cette fonction effectue une itération de la machine de Turing, en suivant les règles associées à la couleur de la LED courante. Elle s'assure aussi de la validité des états et des positions.

3.4.1.3 *Déplacement de la tête*

move_head

Effectue le déplacement de la tête de lecture dans une direction donnée, en s'assurant que la position reste dans les limites du ruban.



3.4.2 Module de Parsing

3.4.2.1 *Objectif*

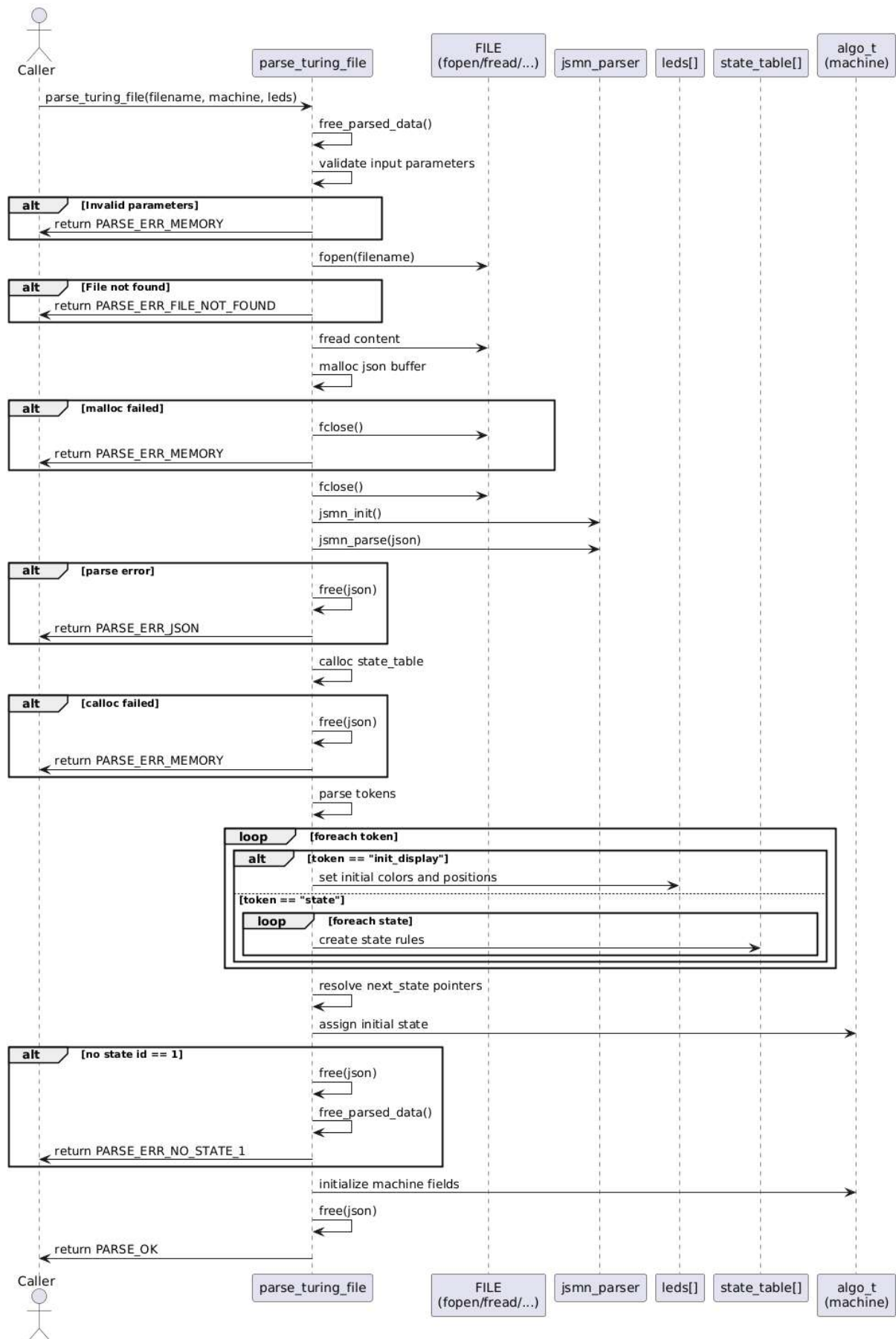
Le module de parsing est responsable de :

- Charger un fichier JSON décrivant la configuration de la machine.
- Extraire les paramètres– d’initialisation (position initiale, mouvement initial, couleur des LEDs).
- Créer dynamiquement une table d’états avec les règles de transition pour chaque couleur.
- Résoudre les références d’états pour passer de simples identifiants à des pointeurs vers les structures mémoire correspondantes.

3.4.2.2 *Fonction principale*

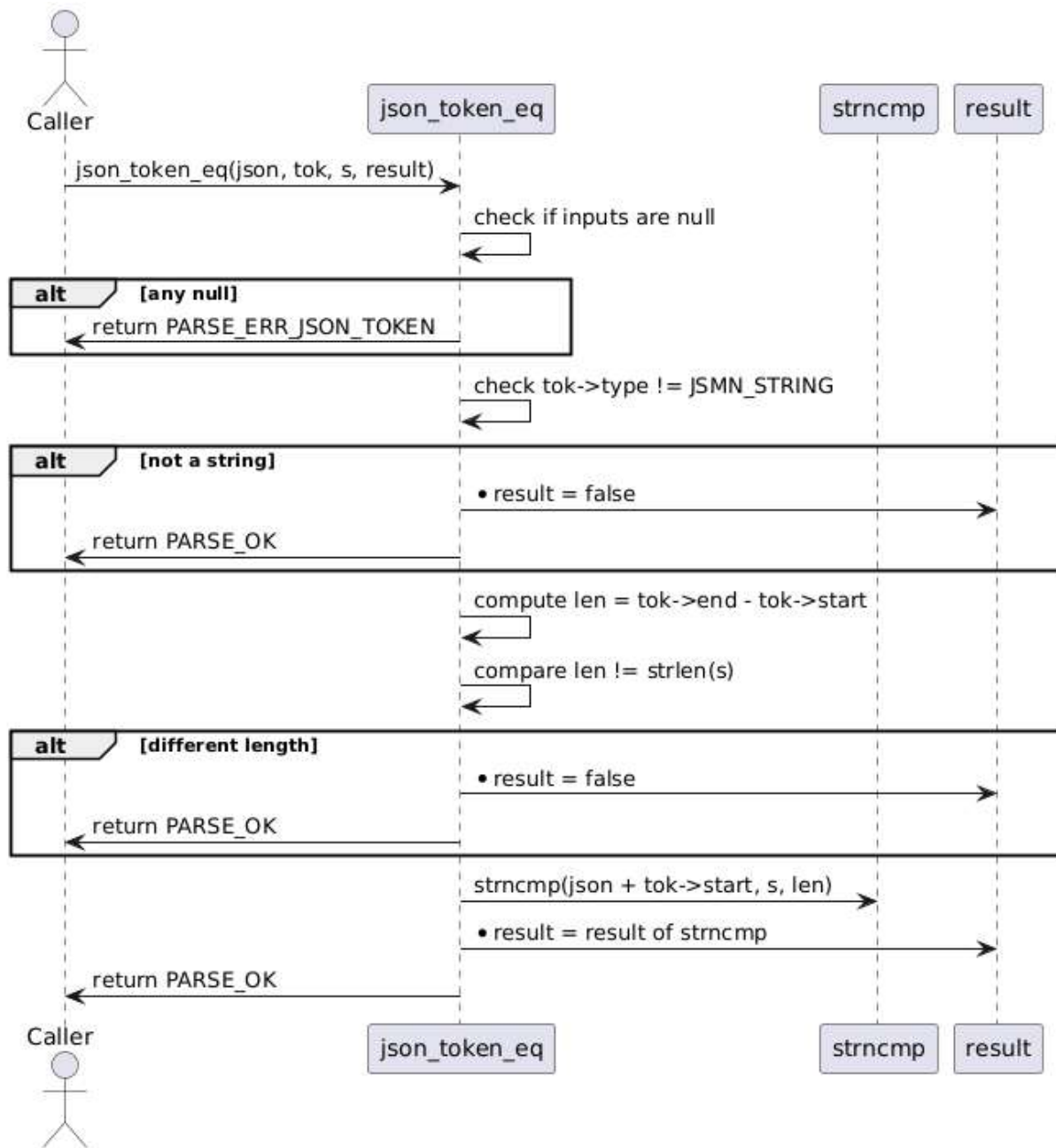
parse_turing_file

Fonction centrale qui orchestre l'ensemble des étapes du parsing.

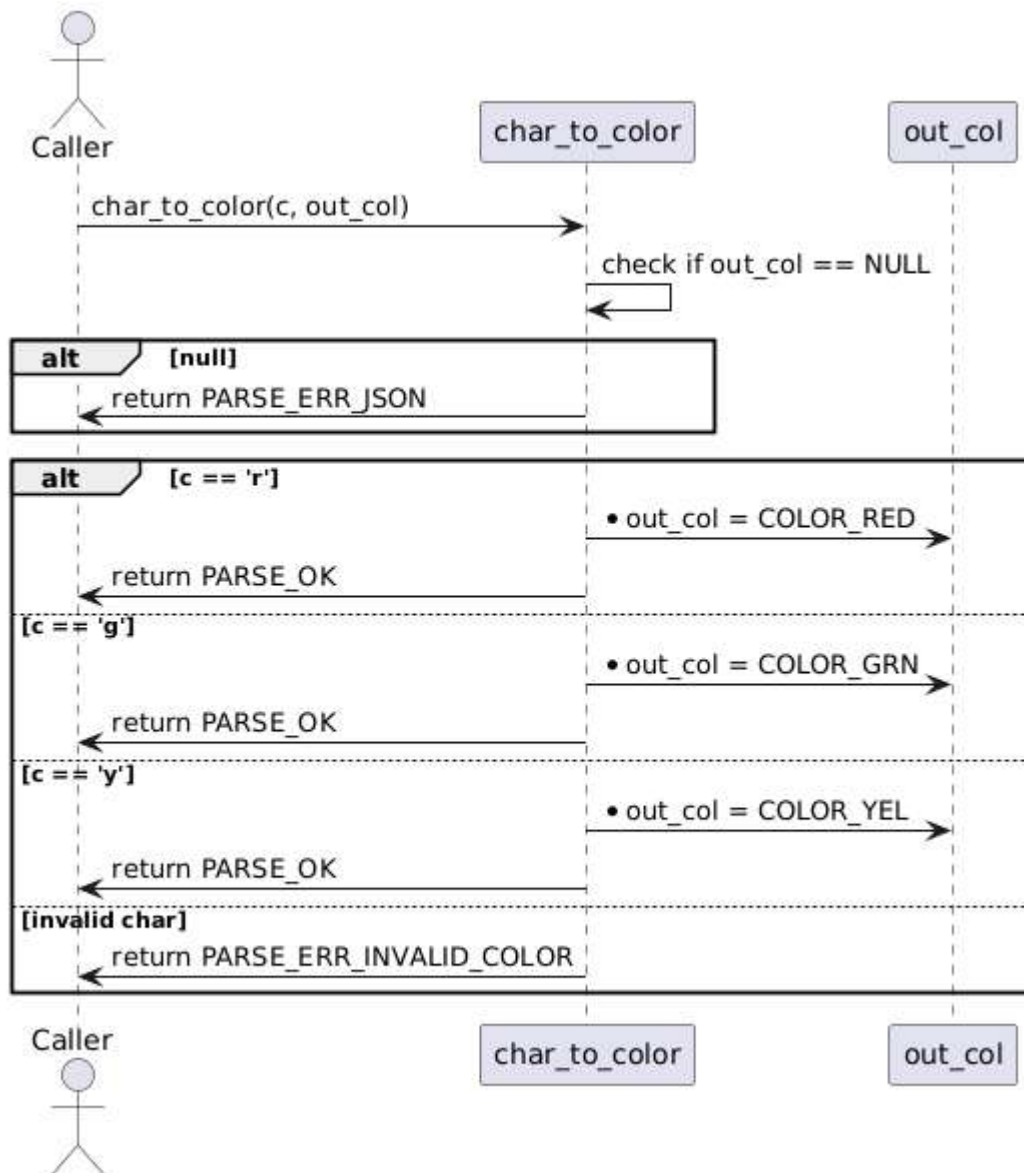


3.4.2.3 Fonctions utilitaires

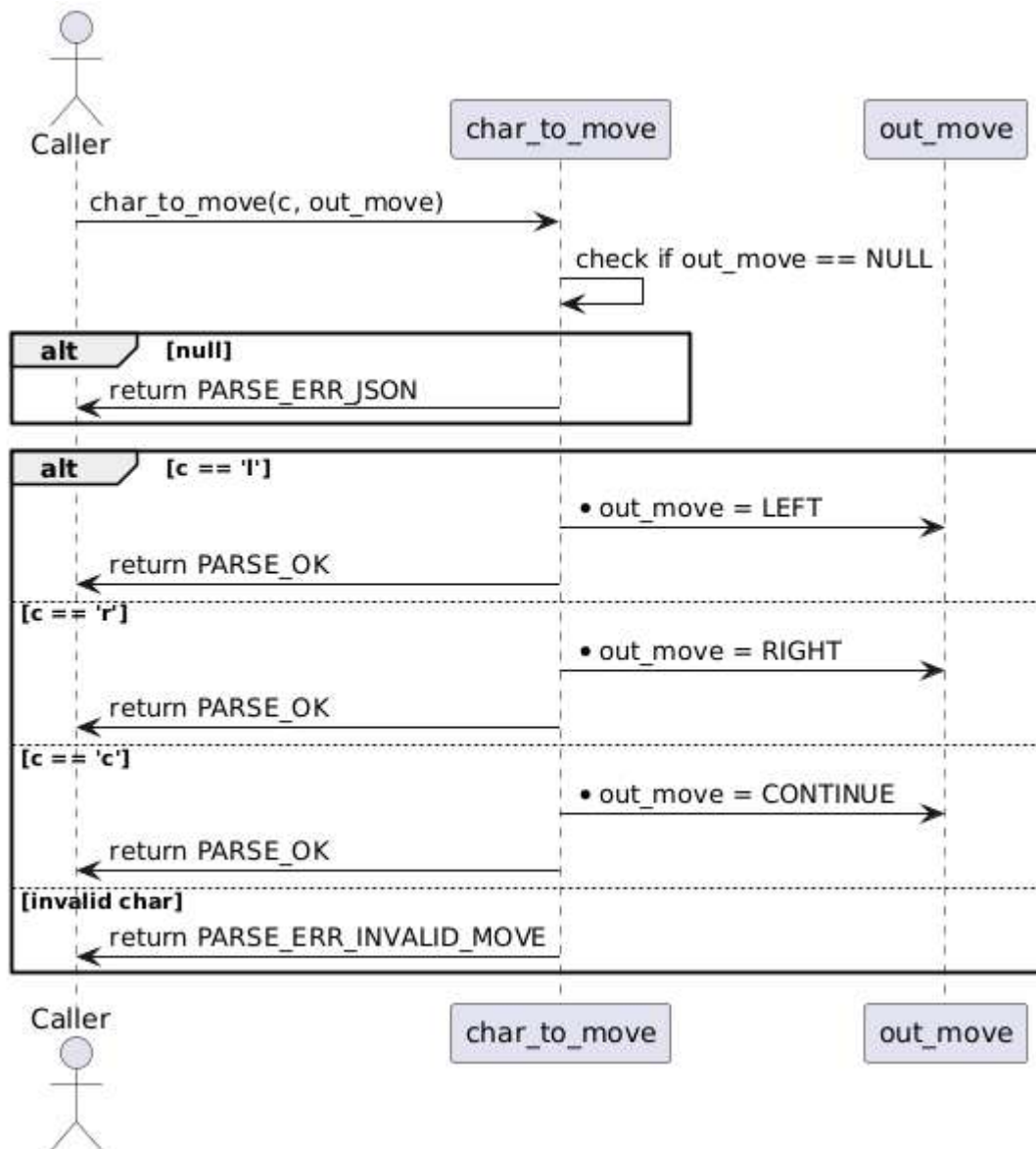
- **json_token_eq** : Compare une clé JSON à une chaîne attendue.



- **char_to_color** : Convertit un caractère en une couleur interne (color_t).



- **char_to_move** : Traduit un caractère en une direction de mouvement (move_t).



3.4.3 Gestion des erreurs

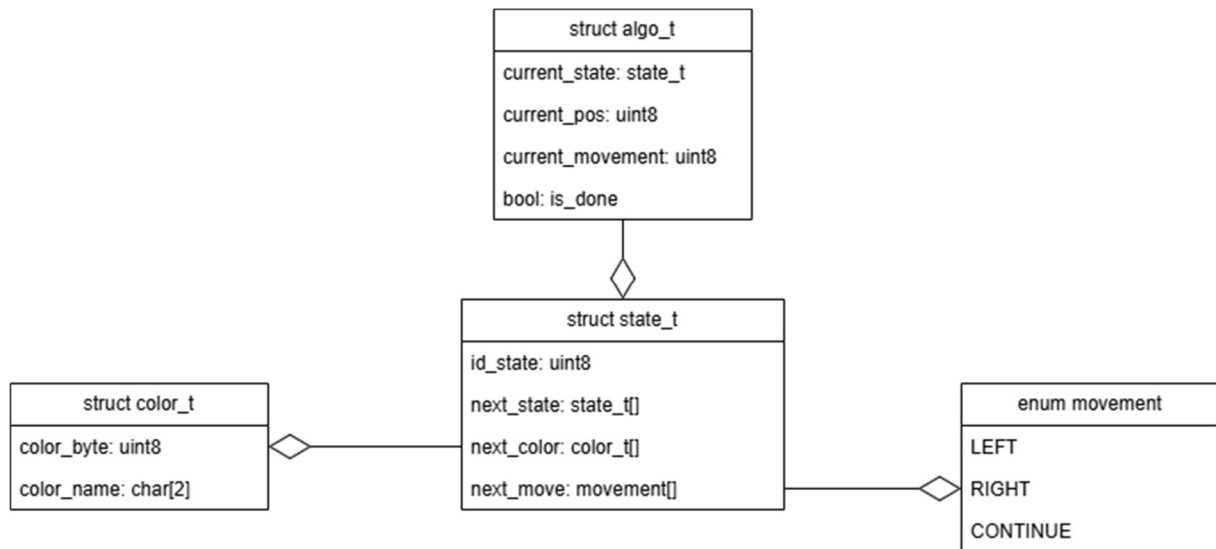
Chaque fonction retourne des codes d'erreurs précis (`PARSE_ERR_XXX`, `ALGO_ERR_XXX`), permettant un diagnostic clair en cas de problème :

- Fichier introuvable.
- Mémoire insuffisante.
- Format JSON invalide.
- État inexistant ou mal relié.
- Débordement du ruban.

3.4.4 Amélioration possible

La machine de Turing implémentée ici est hautement configurable via un fichier JSON, tout en respectant des principes de robustesse, de modularité, et de clarté. Le découpage en modules distincts permet d'assurer une séparation claire entre la configuration (parsing) et l'exécution (algorithme).

Cependant, nous aimerions faire en sorte que cette machine puisse prendre une quantité configurable de couleur et d'état, c'est pour cela qu'une révision des structures a été faite pour en utilisant des tableaux pour chaque variable.



4 Synthèse

Ce projet de Machine de Turing nous a permis de mettre en action les différentes connaissances que nous avons apprises au cours des leçons d'AMIIC et de MCU de M. ROLLAND. Nous avons pris le temps d'étudier les SF1D ainsi que de les corriger par nous-même tous comme les analyses de fonctions. Ce projet été pour nous très attrayant par l'approche informatique surtout dans la programmation et l'approche électronique qui était pour nous une totale nouveauté étant donné que chacun de nous venait de deux cursus accés autre que l'électronique (PeiP Maths pour Yassine et BUT Informatique pour Alexis)

La gestion du projet était dans un premier temps en très bonne posture car nous nous étions scindé les rôles en deux. Yassine s'occupait de la partie hardware et électronique tandis que Alexis s'occupait de la partie programmation. Tout en tenant au courant l'autre de son avancé et en lui demandant de l'aide si besoin.

Le souci dans la gestion de ce projet malheureusement a été la gestion de temps. Quand dans un premier temps, il y a eu une bonne lancé avec des réunions avec l'encadrant et une envie de faire ce projet, dans un second temps les cours ont vite pris le dessus et la gestion des emplois du temps n'a pas été bien faite pour avancer dans le projet ce qui l'a retardé. C'est une erreur que nous avons effectuée et qui nous a permis d'apprendre que la récurrence dans le travail est un aspect important dans un projet. Récurrence que l'on n'oubliera pas de mettre en place sur les projets des futures années.

Ce projet nous a permis de prendre en main de manière concrète les logiciels MultiSim (pour le circuit électronique avec la création des composants via le wizard), Ultilab (pour le dessin du circuit PCB qui n'a pas abouti) et MPLab X IDE pour la programmation du microcontrôleur.

5 Remerciements

Dans ce projet, nous tenons à remercier notre encadrant M. Ronan BOCQUILLON qui nous a accompagné et qui essayait de se rendre disponible le plus rapidement possible lorsqu'on en ressentait le besoin.

Nous remercions M. Alexis ROLLAND pour ses cours d'AMIIC et de MCU qui nous ont aidé pour la partie recherche et programmation du projet, son aide aussi pour les fournitures notamment du PICKit 3 pour la programmation du microcontrôleur. Nous le remercions aussi pour les cours d'électronique auxquels nous avons assisté au début qui nous ont permis de répondre à des questions pour le projet ainsi que de la reprise des notions de cours pour mieux avancer par la suite.

Nous remercions M. Sébastien BEAUFILS qui nous a fourni en équipement, notamment sur l'alimentation 3.3V et 5V qui nous a beaucoup aidé pour ce projet.

Nous remercions aussi Clément ALLEGRE-COMMINGES et Romain BROUARD pour leur aides précieuses sur la compréhension de ce qu'ils avaient fait dans le passé sur ce projet et les réponses fournies aux diverses questions.

Nous terminerons en remerciant aussi les élèves de la classe des 3A ISIE qui nous ont aidé sur des questions dans l'aspect électronique du projet (notamment Eliot PARAT et Anes HERZI).

Bibliographie

https://fr.wikipedia.org/wiki/Machine_de_Turing

Datasheet fournies en annexes